

# Brain Tumor Detection

## About Brain Tumor Detection

The objective of our project is to develop a brain tumor detection system using python. Python contains libraries like tensorflow, keras, PIL etc which can be used to develop a program to train a machine learning model to detect brain tumors from MRI images.

The Brain Tumor Detection system is a deep learning solution which gives high accuracy results without manual diagnosis.

## Prerequisites

This ML project requires knowledge about machine learning, image processing with the python libraries like Tensorflow, keras and PIL which are useful for making a deep learning application. These libraries can be downloaded easily into a python environment and the model can be trained easily.

## Project File Structure

Steps to build a Brain Tumor Detection system using CNN:

- Import all the necessary libraries and modules.
- Loading and visualizing data
- Preprocessing images
- Making the model architecture
- Evaluating model on test dataset and saving the trained model
- Taking input images and predicting brain tumor

## Download Brain Tumor Detection Project Code

Source code: [🔗 Tumor\\_hai?.ipynb](#)

## 1. Import necessary libraries

```
import kagglehub
import os
import numpy as np
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Flatten, Dropout
from tensorflow.keras.preprocessing.image import load_img
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import VGG16
from sklearn.utils import shuffle
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import label_binarize
import seaborn as sns
from PIL import Image, ImageEnhance
```

### Explanation:

- `kagglehub` imports Kaggle API to access datasets
- `os` imports operating system
- `random` generates random numbers
- `numpy` handles numeric operations
- `from tensorflow.keras.models import Sequential` helps develop neural network layer by layer
- `from tensorflow.keras.layers import Input, Dense, Flatten, Dropout` imports keras layers to define shape, dense layers, flatten layer and dropout
- `from tensorflow.keras.preprocessing.image import load_img` to load image files.

- `from tensorflow.keras.applications import VGG16` imports VGG16 model for transfer learning
- `from tensorflow.keras.optimizers import Adam` imports adam optimizer to train model
- `from tensorflow.keras.models import Sequential` modules help us build, train and compile model.
- `PIL` helps to load and modify imgs
- `matplotlib` and `seaborn` to plot graphs & confusion matrix
- `sklearn.metrics` to generate reports and evaluation metrics

## 2. Load and visualize data

```
path =
kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-dataset")
train_dir = f'{path}/Training'
test_dir = f'{path}/Testing'
```

```
train_paths = []
train_labels = []
for label in os.listdir(train_dir):
    for image in os.listdir(os.path.join(train_dir, label)):
        train_paths.append(os.path.join(train_dir, label, image))
        train_labels.append(label)
train_paths, train_labels = shuffle(train_paths, train_labels)
```

#Visualization

```
random_indices = random.sample(range(len(train_paths)), 10)
fig, axes = plt.subplots(2, 5, figsize=(15, 8))
axes = axes.ravel()
for i, idx in enumerate(random_indices):
    img_path = train_paths[idx]
```

```

img = Image.open(img_path)
img = img.resize((224, 224))
axes[i].imshow(img)
axes[i].axis('off')
axes[i].set_title(f"Label: {train_labels[idx]}", fontsize=12)
plt.tight_layout()
plt.show()

```

## Explanation:

- The dataset is separated into Training and Testing. These are used to train and evaluate the model.
- Paths and labels are shuffled to prevent model bias.
- We plot a random sample of training images to confirm that the images are loaded correctly.

## 3. Image Preprocessing

```

#image augmentation
def augment_image(image):
    image = Image.fromarray(np.uint8(image))
    image = ImageEnhance.Brightness(image).enhance(random.uniform(0.8,
1.2))
    image = ImageEnhance.Contrast(image).enhance(random.uniform(0.8,
1.2))
    image = np.array(image)/255.0
    return image

#loading images
def open_images(paths):
    images = []
    for path in paths:
        image = load_img(path, target_size=(image_size, image_size))
        image = augment_image(image)
        images.append(image)
    return np.array(images)

```

```

#encoding labels
def encode_labels(labels):
    unique_labels = sorted(os.listdir(train_dir))
    encoded = [unique_labels.index(label) for label in labels]
    return np.array(encoded)

#data generator for batching
def datagen(paths, labels, batch_size=12, epochs=1):
    for _ in range(epochs):
        for i in range(0, len(paths), batch_size):
            batch_paths = paths[i:i+batch_size]
            batch_images = open_images(batch_paths)
            batch_labels = labels[i:i+batch_size]
            batch_labels = encode_labels(batch_labels)
            yield batch_images, batch_labels

```

## Explanation:

Preprocessing includes the following:

- **Augmentation:** Improves training by variation in brightness and contrast.
- **Resizing & Normalizing:** Images are resized to 128x128 pixels and scaled to 0 to 1 for better performance.
- **Label Encoding:** Converts folder name into numeric labels.
- **Data Generator:** It yields data in batches to avoid memory overload.

## 3. Model Architecture

```

image_size = 128
base_model = VGG16(input_shape=(image_size, image_size, 3),
include_top=False, weights='imagenet')
for layer in base_model.layers:
    layer.trainable = False

```

```

base_model.layers[-2].trainable = True
base_model.layers[-3].trainable = True
base_model.layers[-4].trainable = True

model = Sequential()
model.add(Input(shape=(image_size, image_size, 3)))
model.add(base_model)
model.add(Flatten())
model.add(Dropout(.3))
model.add(Dense(128, activation="relu" )
model.add(Dropout(.2))
model.add(Dense(4, activation="softmax"))
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy',
metrics=['sparse_categorical_accuracy'])

batch_size = 20
steps = int(len(train_paths)/batch_size)
epochs = 5
history = model.fit(datagen(train_paths, train_labels,
batch_size=batch_size, epochs=epochs), epochs=epochs,
steps_per_epoch=steps,)

```

## Explanation:

- **VGG16** for **transfer learning**, for better accuracy.
- Most VGG16 layers are frozen and we're using just a few for better performance on mri images.
- Customised **Dense** and **Dropout** layers for 4-class problem.
- The final layer uses **softmax** activation for multiclass classification.
- Model compiled using **Adam** optimizer and **sparse categorical crossentropy** loss.

This architecture makes the model lightweight yet powerful, getting features from the VGG16.

- Defining batch size and number of epoch based on training set size.
- Training history stores loss and accuracy for each epoch.

### 3. Evaluation Results

```
test_images = open_images(test_paths)
test_labels_encoded = encode_labels(test_labels)
test_predictions = model.predict(test_images)
#Classification Report
print("Classification Report:")
print(classification_report(test_labels_encoded,
np.argmax(test_predictions, axis=1)))
```

```
#Confusion Matrix
conf_matrix = confusion_matrix(test_labels_encoded,
np.argmax(test_predictions, axis=1))
print("Confusion Matrix:")
print(conf_matrix)
plt.figure(figsize=(8, 6))
sorted_labels = sorted(os.listdir(train_dir))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Reds",
xticklabels=sorted_labels, yticklabels=sorted_labels)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("Real Labels")
plt.show()
```

### Explanation:

- Plotting **accuracy and loss** to check for overfitting or underfitting.

- `classification_report` to view **precision, recall, f1-score** for tumor types.
- **Confusion matrix** to show how well the model is classifies each category.

This helps us measure actual performance of the model.

### 3. Saving and Using the Model

```
model.save('model.h5')
model = load_model('model.h5')
class_labels = sorted(os.listdir(train_dir))

def detect_and_display(img_path, model, image_size=128):
    try:
        img = load_img(img_path, target_size=(image_size, image_size))
        img_array = img_to_array(img) / 255.0
        img_array = np.expand_dims(img_array, axis=0)
        predictions = model.predict(img_array)
        predicted_class_index = np.argmax(predictions, axis=1)[0]
        confidence_score = np.max(predictions, axis=1)[0]
        if class_labels[predicted_class_index] == 'notumor':
            result = "No Tumor"
        else:
            result = f"Tumor: {class_labels[predicted_class_index]}"

        plt.imshow(load_img(img_path))
        plt.axis('off')
        plt.title(f"{result} (Confidence: {confidence_score *
100:.2f}%)")
        plt.show()
    except Exception as e:
        print("Error processing the image:", str(e))
```

### Explanation:

- After training, the model is saved to import to other app or webpage.

- We can run predictions on user uploaded MRI scans to detect the tumor type.
- The prediction result is displayed with a confidence score using `matplotlib`.

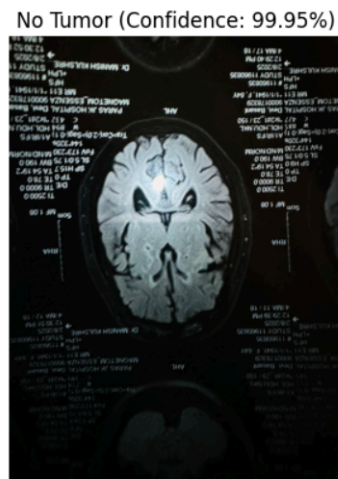
## Output

```
[ ] from google.colab import files
    uploaded = files.upload()
```

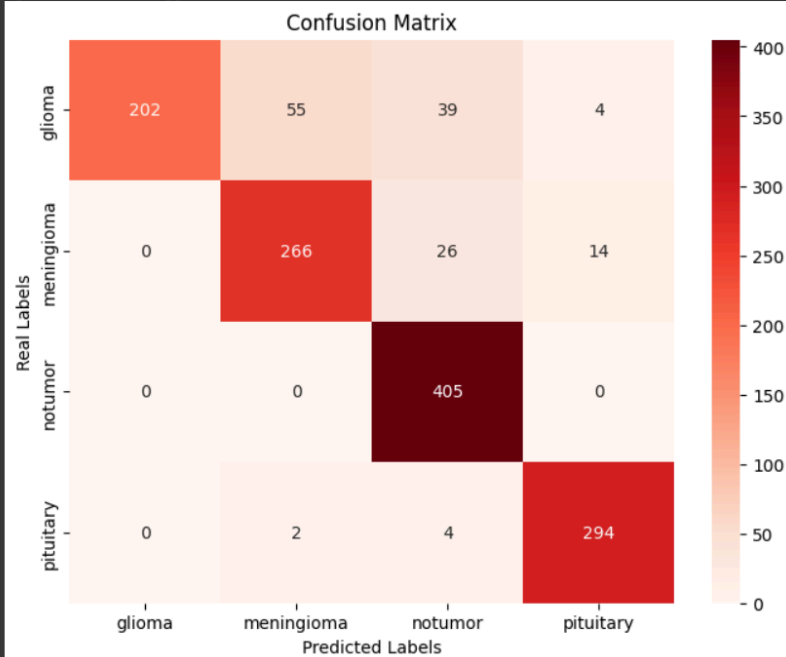
Choose Files No file chosen Upload widget is only  
Saving mri.jpg to mri (1).jpg

```
from PIL import Image
for filename in uploaded.keys():
    detect_and_display(filename, model)
```

1/1 0s 36ms/step



```
Confusion Matrix:
[[202  55  39   4]
 [   0 266  26  14]
 [   0   0 405   0]
 [   0   2   4 294]]
```



41/41

4s 83ms/step

| Classification Report: |           |        |          |         |  |
|------------------------|-----------|--------|----------|---------|--|
|                        | precision | recall | f1-score | support |  |
| 0                      | 1.00      | 0.67   | 0.80     | 300     |  |
| 1                      | 0.82      | 0.87   | 0.85     | 306     |  |
| 2                      | 0.85      | 1.00   | 0.92     | 405     |  |
| 3                      | 0.94      | 0.98   | 0.96     | 300     |  |
| accuracy               |           |        | 0.89     | 1311    |  |
| macro avg              |           |        | 0.88     | 1311    |  |
| weighted avg           |           |        | 0.89     | 1311    |  |

## Project Applications

The brain tumor detection system can be used for:

- **Early Diagnosis:** Assisting doctors in detecting tumors at an early stage
- **Radiology Automation:** Reducing workload of analysts by analyzing MRI scans automatically
- **Rural Healthcare:** Supporting diagnosis in remote areas with limited access to specialists
- **Second Opinions:** Providing additional insight in critical cases for better decision-making
- **Medical Research:** Helping researchers analyze large MRI datasets efficiently

## Superiority Over Traditional Diagnostic Methods

This brain tumor detection system offers major advantages over traditional/manual diagnosis methods:

- **Speed & Efficiency:** Analyzes hundreds of MRI scans in seconds, reducing diagnosis time drastically.
- **High Accuracy:** Uses deep learning to detect subtle tumor patterns that may be missed by the human eye.
- **Consistency:** Eliminates human errors due to fatigue or bias, giving reliable results every time.
- **Cost-Effective:** Reduces the need for multiple expert opinions and speeds up screening in hospitals.
- **Scalable:** Can be deployed in multiple hospitals or clinics without needing extra manpower.
- **Remote Use:** Useful in telemedicine or rural health centers where radiologists aren't available.

- **Smart Assistance:** Works as a supportive tool for doctors—not a replacement—enhancing medical judgment.

By combining deep learning with medical imaging, this system brings the **power of automation, accuracy, and accessibility** to brain tumor diagnosis—making it a game-changer in healthcare.

+

## Conclusion

The brain tumor detection system successfully showcases the potential of deep learning in medical imaging. By leveraging a pre-trained VGG16 model, the project achieves high accuracy in classifying different types of brain tumors from MRI scans. Its ability to deliver fast, reliable results makes it ideal for real-world healthcare applications. With minimal hardware requirements and scalable architecture, this solution is both efficient and practical for deployment in hospitals, diagnostic labs, and remote health centers.